

So how do you run QML applications on Qt5 on systems where there's no 3D support? Well, QtQuick 1.0 still ships with Qt5, so you can use that. Or you can drop to using standard C++ GUIs, which can fall back to 2D rendering if 3D isn't available on the system.

So that's enough of theory. Let's build something.

Starting off

Open up QtCreator and start a new project. This time, choose Qt Quick Application (and not Qt Quick UI, which creates a QML-only application with no C++ code). Hit *Choose*, set the location and hit *Next*.

Now you'll be presented with a drop-down list of component sets to use for the application. You'll need to use the latest available Qt Quick Controls (at the time of writing, this is Qt Quick Controls 1.3). Qt Quick Controls is a set of desktop application widgets implemented on top of raw Qt Quick 2 (which is also available if you want to use it). Once you're done, hit *Next*.

Run through the rest of the wizard. Make sure *Desktop* is selected when you're selecting the kits to use for the application. Check the summary, and then finish the wizard to get started with the code.

QML

At this point, you'll be dropped into the editor with *main.qml* open. You'll be able to inspect this file to figure out what QML applications look like. But let me help you out.

At the top of the code, you'll see a bunch of import statements, like in Python. These lines import the QML components that you'll be using to write your applications. In the sample *main.qml* that opens up, the import lines are:

```
import QtQuick 2.4
import QtQuick.Controls 1.3
import QtQuick.Window 2.2
import QtQuick.Dialogs 1.2
```

So first, QtQuick 2 itself is imported, followed by a bunch of extensions, including *Controls*, which implement GUI widgets; *Dialogs*, which are used to create pop-up dialogue boxes, and *Window*, which is used to create the main application window in QML.

QML components can be written in QML (remember that it's basically JavaScript; it's actually powered by Google's V8, so it's a full-blown programming language to work in) or C++. We're going to write a component in C++ to make the QML GUI talk to the fortune cookie server.

The rest of the code describes the GUI. It's very much like HTML. There's one root element (in this case, an *ApplicationWindow*, which encapsulates the rest of the element hierarchy). Notice that all elements look exactly like JSON objects, with properties that can either be other objects (like strings or numbers) or JavaScript functions.

Once you've figured out what QML looks like (don't

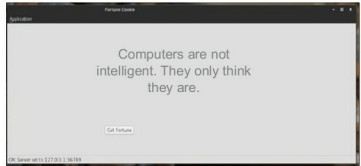


Figure 1: Fortune cookie

worry if you still don't understand how to write code yourself), we're ready to start building our own GUI.

Building the GUI

The first thing you should do is delete the *MainForm.ui.qml* file in the *Projects* pane. We won't be needing it since we'll be writing the entire UI inside *main.qml*.

Now comes the difficult bit. *main.qml* in our program is 150 lines long, which is a tad too long to print out in this magazine. So we're going to have to do it a little differently.

First of all, you're going to have to fetch all the source code for this article. It's available on my GitHub account, at <https://github.com/BaloneyGeek/FortuneClientExample>. Like the last article, this is a fully functioning application, so you can just clone the repository, build it and run it.

But let's see some of the code at least. We start with the imports:

```
import QtQuick 2.4
import QtQuick.Layouts 1.1
import QtQuick.Controls 1.3
import QtQuick.Window 2.2
import QtQuick.Dialogs 1.2

import Fortune 1.0
```

We're importing a whole bunch of QtQuick extensions, including *Layouts*, which we'll use to build the layout of the GUI (as the name suggests).

The last thing we're importing—Fortune 1.0—does not quite exist yet, so QtCreator is going to give it a red underline. We'll build this component in C++ later.

The next few lines set up the application window, and populate the menu bar and the status bar. We won't reproduce the code in print, but it's simple enough and one look will give you an idea of what's going on. What you'll find interesting is that there are ampersand characters in some of the strings. This is to set up keyboard shortcuts—the letter right after the ampersand symbol becomes the keyboard shortcut for that item.

Then we come to the main UI form of the program. The UI features one giant label in the centre top part of the window displaying the fortune cookie, and one button at the centre bottom that can be clicked to attempt to get a